

or gateway—if we want to access the local machine from outside the network, it is impossible to access it directly. But by forwarding the local ports to a remote host, we can access the local machine through ports of the remote host.

Let's see how remote port forwarding is executed:

```
ssh -R remoteport:remotehost:localhost
```

For example:

```
ssh -R 2020:slynux.org 22
```

To SSH to the local machine from outside the internal network, we can make use of *slynux.org* as *ssh slynux.org:2020*.

SOCKS4 proxy

SSH has an interesting feature called Dynamic Port forwarding with which the SSH TCP connection will work as a SOCKS4 proxy. By connecting to the given port, it handles SOCKS data transfer requests.

An important application of Dynamic Port forwarding is the following case.

Let's suppose you have a machine on a network that is connected to the Internet and you have another machine on the same network that does not have any Internet connection. By using SSH Dynamic port forwarding, you can easily access the Internet by setting up the machine with an Internet connection to act as the SOCKS4 proxy using an SSH tunnel.

For dynamic port forwarding, use the following command:

```
ssh -D 3000 remotehost
```

Now, in your browser, specify proxy settings as:

- SOCKS4
- host: localhost
- port: 3000

To enable the DNS service in Firefox, navigate the *about:config* page and set...

```
network.proxy.socks_remote_dns = true
```

Automatic key authentication

Each time you access the other machine for the remote execution of some command, it probes for the password. This is not desirable when we need to automate tasks. If we need to shut down or reboot all the machines on the LAN, it is impractical to type the user password for command execution on each of the machines. There should be some mechanism that handles automatic authentication without probing for a password.

The solution for this hurdle is public key authentication, for which we will generate a public key from the machine we need to execute remote commands. That public key will be copied to each of the remote machines. Thus each time when we execute remote commands, it will perform a user authentication by verifying the public key and it no more

probes for the passwords.

Generate the public key as follows:

```
slynux@slynux-laptop:~$ ssh-keygen -t rsa
Generating public/private rsa key pair:
Enter file in which to save the key (/home/slynux/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/slynux/.ssh/id_rsa.
Your public key has been saved in /home/slynux/.ssh/id_rsa.pub
The key fingerprint is:
0e:04:3d:e3:2a:54:8c:47:ae:10:9a:96:41:be:c1:8f slynux@slynux-laptop
```

Now we have the public key in the file *~/ssh/id_rsa.pub*.

```
slynux@slynux-laptop:~$ cat .ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAABIwAAAQEAuJ6N7/juQ8CUdmFP816Xn4EI
j73pO7+uHPqBF2Gxg8yeY2mU7dBJUAc5Xx/NhRf7YytozhvWk+n92DBFL6
U62lrukqB/Wd2RHh2w1JH4adC3hCD5Uglaxt5WoZK4nPzGRcbdtTbXc2tELQu
u9e9qowzQ8bU3Wd8UK0+U0/u8XS5WxvWE4W27HIA/WP7Rjp+KDK33Ms9u
IYyx/h3Tv5voP5xV6cYB2h5kMz5oYDBCUpua6uH4v4zDUFNnN6SdpI3213FY/
cGRV71vBCRD5mQd0Xq2hU8npCfz0rQXqGPuuzfVW8le6yRQQPtqXc3/
J5UMGmgungDgw== slynux@slynux-laptop
```

To implement auto authentication, append the public key in the *~/ssh/authorized_keys* file in each of the remote machines where we need to perform auto authentication.

Appending the key can be performed manually or it can be automated using an *ssh* command as follows:

```
ssh remote_host "cat >> .ssh/authorized_keys" < ~/ssh/id_rsa.pub
```

Finally, let us write a single loop shell script to reboot all the switched-on machines in the network.

```
#!/bin/bash
base_ip="192.168.0.":

for machine in $(base_ip{1..255});
do
    ping -c2 $machine && /dev/null;
    if [ $? -eq 0 ];
    then
        ssh $machine reboot;
    fi
done
```

That's it about the secure shell. Hope you enjoyed this tutorial. Till we meet again, happy hacking! 

By: Sarath Lakshman

The author is a Hactivist of Free and Open Source Software from Kerala. He loves working on the GNU/Linux environment and contributes to the PitiVi video editor project. He is also the developer of SLYNIX, a distro for newbies. He blogs at www.sarathlakshman.info